

Material de Estudio: Bases de Datos Relacionales y SQL Server

Profesor: Jorge Izaguirre **Materia:** Programación .NET **Curso:** 6to Año - TÉCNICO EN PROGRAMACIÓN

Parte 1: Introducción - ¿Por Qué Bases de Datos? El Mundo Relacional

(1.1) La Necesidad de Organizar la Información

Imagina intentar gestionar la información de todos los alumnos de la escuela usando archivos de texto separados o una única planilla de cálculo gigante. Pronto te enfrentarías a problemas:

- **Redundancia:** Datos repetidos en muchos lugares (ej: el domicilio de un alumno en múltiples registros de inscripción).
- **Inconsistencia:** Si actualizas un dato en un lugar (ej: cambias un apellido), ¿lo actualizas en todos los demás sitios donde aparece? Si no, los datos se vuelven inconsistentes.
- **Dificultad de Acceso:** ¿Cómo encontrar rápidamente todos los alumnos que viven en Merlo? ¿O los que aprobaron Programación .NET el año pasado? Sería muy lento y complejo.
- **Problemas de Integridad:** ¿Cómo te aseguras de que cada alumno tenga un legajo único? ¿O que una nota siempre esté entre 0 y 10?
- **Acceso Concurrente:** ¿Qué pasa si dos personas intentan modificar la misma información al mismo tiempo?

Para solucionar estos y otros problemas, utilizamos **Bases de Datos (DB)**.

- **¿Qué es una Base de Datos?** Es una colección organizada de datos estructurados, almacenados electrónicamente de forma que se puedan consultar, gestionar y actualizar eficientemente.
- **¿Qué es un Sistema Gestor de Bases de Datos (DBMS - Database Management System)?** Es el *software* que nos permite crear, administrar y interactuar con la base de datos. Actúa como intermediario entre el usuario (o la aplicación) y los datos físicos almacenados. Ejemplos: Microsoft SQL Server, Oracle Database, MySQL, PostgreSQL, SQLite. Nosotros nos centraremos en **Microsoft SQL Server**.

(1.2) El Modelo Relacional: Orden y Estructura

Existen diferentes modelos para organizar los datos (jerárquico, en red, orientado a objetos, documental/NoSQL), pero el **Modelo Relacional** ha sido el dominante durante décadas y sigue siendo fundamental.

- *¿Por qué es tan popular?*
 - **Estructura Simple y Clara:** Organiza los datos en tablas bidimensionales fáciles de entender.
 - **Consistencia:** Utiliza mecanismos (claves, restricciones) para mantener la integridad y validez de los datos.
 - **Flexibilidad:** Permite consultar los datos de formas muy variadas y complejas.
 - **Lenguaje Estándar:** Utiliza SQL (Structured Query Language) como lenguaje estándar para interactuar con los datos.
- **Componentes Clave del Modelo Relacional:**
 - **Tablas (o Relaciones o Entidades):** Son el componente principal. Almacenan datos sobre un tipo específico de "cosa" o *entidad* (ej: *Alumnos*, *Materias*, *Productos*, *Facturas*). Se visualizan como una grilla con filas y columnas.

- **Columnas (o Atributos o Campos):** Representan las características o propiedades de la entidad que queremos almacenar. Cada columna tiene un nombre único dentro de la tabla y un **Tipo de Dato** específico. (ej: en la tabla *Alumnos*, podríamos tener columnas como *Legajo*, *Nombre*, *Apellido*, *FechaNacimiento*, *Promedio*).
- **Filas (o Tuplas o Registros):** Representan una instancia individual o un registro específico de la entidad. Cada fila contiene un valor para cada columna de la tabla (o un valor *NULL*). (ej: una fila en la tabla *Alumnos* representaría a un alumno concreto con su legajo, nombre, etc.).

(1.3) Tipos de Datos en SQL Server: La Naturaleza de la Información

Al definir una columna, debemos especificar qué tipo de dato almacenará. Elegir el tipo correcto es crucial.

- *¿Por qué es importante?*
 - **Integridad:** Asegura que solo se almacenen datos válidos (no puedes poner texto en una columna numérica).
 - **Almacenamiento Eficiente:** Cada tipo de dato ocupa un espacio diferente en disco y memoria. Usar el tipo adecuado ahorra espacio.
 - **Rendimiento:** Las operaciones son más rápidas cuando se realizan sobre el tipo de dato correcto.
- **Tipos de Datos Comunes en SQL Server:**
 - **Numéricos Exactos:**
 - *INT*: Números enteros (aprox. -2 mil millones a +2 mil millones). Para IDs, contadores, etc. (También *BIGINT*, *SMALLINT*, *TINYINT* para rangos diferentes).
 - *DECIMAL(p, s)* o *NUMERIC(p, s)*: Números con una cantidad fija de dígitos totales (*p*, precisión) y dígitos decimales (*s*, escala). **Ideal para dinero** y cálculos exactos. Ej: *DECIMAL(10, 2)* puede almacenar hasta 99,999,999.99.
 - **Numéricos Aproximados:**
 - *FLOAT*, *REAL*: Números con decimales de punto flotante. Son aproximados, pueden tener pequeños errores de redondeo. Usar para mediciones científicas, evitar para dinero.
 - **Cadenas de Caracteres (Texto):**
 - *VARCHAR(n)*: Texto de longitud variable hasta *n* caracteres. Ocupa solo el espacio de los caracteres guardados + un pequeño overhead. Para nombres, descripciones cortas, etc.
 - *NVARCHAR(n)*: Igual que *VARCHAR* pero almacena caracteres **Unicode** (soporta múltiples alfabetos y símbolos). Ocupa el doble de espacio por carácter. Usar si necesitas soporte internacional.
 - *CHAR(n)*: Texto de longitud *fija* de *n* caracteres. Si guardas menos, se rellena con espacios. Útil para códigos de longitud fija (ej: CUIT/CUIL si siempre tienen el mismo formato).
 - *TEXT*, *NTEXT* (obsoletos, usar *VARCHAR(MAX)*, *NVARCHAR(MAX)* para textos muy largos).
 - **Fecha y Hora:**
 - *DATE*: Almacena solo la fecha (año, mes, día).
 - *TIME*: Almacena solo la hora (hora, minuto, segundo, fracciones).
 - *DATETIME2(p)*: Almacena fecha y hora con precisión de fracciones de segundo configurable (*p*). Es el tipo recomendado actualmente sobre el antiguo *DATETIME*.
 - **Booleanos:**
 - *BIT*: Almacena un valor 0 (falso) o 1 (verdadero). Ocupa muy poco espacio. Ideal para indicadores sí/no.
 - **Otros:** *BINARY* (datos binarios), *XML*, *GEOGRAPHY*, *GEOMETRY*, etc.

(1.4) El Significado de **NULL**

En bases de datos, **NULL** **no** significa cero, ni una cadena vacía, ni falso. **NULL** representa la **ausencia de un valor conocido o aplicable**.

- *¿Cuándo se usa?*
 - El valor es desconocido en este momento (ej: email de un cliente que no lo proporcionó).
 - El valor no aplica para este registro (ej: fecha de baja de un empleado activo).
- *Implicaciones:*
 - Las comparaciones con **NULL** son especiales (ej: **Columna = NULL** no funciona como se espera, se usa **Columna IS NULL** o **Columna IS NOT NULL**).
 - Las funciones de agregación (como **SUM**, **AVG**) suelen ignorar los **NULLs**.
 - Las Claves Primarias **no pueden** ser **NULL**.
 - Puedes especificar si una columna permite **NULLs** o no al crear la tabla (con **NOT NULL**).

(1.5) Claves Primarias (PK): El Identificador Único

Para que una tabla sea útil en el modelo relacional, necesitamos una forma de identificar **inequívocamente** cada fila. Esa es la función de la **Clave Primaria (Primary Key - PK)**.

- *Definición:* Una columna (o un conjunto de columnas) cuyos valores **identifican de forma única** cada fila de la tabla.
- *Reglas Fundamentales:*
 1. **Unicidad:** No puede haber dos filas con el mismo valor en la(s) columna(s) de la PK.
 2. **No Nulidad:** La(s) columna(s) de la PK **no pueden** contener valores **NULL**.
- *¿Por qué son esenciales?*
 - Garantizan que no haya registros duplicados o ambiguos.
 - Permiten localizar y acceder a una fila específica de forma rápida y directa.
 - Son **fundamentales** para establecer relaciones con otras tablas mediante Claves Foráneas.
- **Tipos de PK:**
 - **PK Simple:** Formada por una sola columna (ej: *Legajo* en *Alumnos*, *IDProducto* en *Productos*). Es lo más común.
 - **PK Compuesta:** Formada por la combinación de dos o más columnas. La *combinación* de valores debe ser única. (ej: en una tabla de unión *Inscripciones(IdAlumno, IdMateria)*, la PK podría ser la combinación de ambas columnas).
- **Elección de la PK:**
 - **Claves Naturales:** Atributos que ya existen en el mundo real y son únicos (ej: DNI, CUIT, ISBN). Pueden ser problemáticas si cambian o si no son tan únicas como se pensaba.
 - **Claves Artificiales (o Surrogadas):** Una columna numérica (**INT** o **BIGINT**) añadida específicamente para ser la PK, usualmente con valores autoincrementales (**IDENTITY** en SQL Server). Son la opción más recomendada generalmente porque son estables, siempre únicas y no tienen significado de negocio que pueda cambiar. (ej: *IdCliente*, *IdPedido*).

(1.6) Claves Foráneas (FK): Conectando las Tablas

Las bases de datos relacionales almacenan información sobre diferentes entidades en tablas separadas. Las **Claves Foráneas (Foreign Key - FK)** son el mecanismo que nos permite **enlazar** estas tablas y representar las relaciones entre ellas.

- **Definición:** Una columna (o conjunto de columnas) en una tabla (la tabla "referenciante" o "hija") cuyos valores **deben coincidir** con los valores de la Clave Primaria (PK) de *otra* tabla (la tabla "referenciada" o "padre").
- **Propósito:**
 - **Establecer la relación lógica:** Indican cómo se conectan las filas de una tabla con las filas de otra.
 - **Garantizar la Integridad Referencial:** Es la función más importante. El DBMS se asegura de que:
 - No se pueda insertar una fila en la tabla hija si el valor de su FK no existe como PK en la tabla padre (ej: no puedes crear un *Pedido* para un *Cliente* que no existe).
 - No se pueda eliminar una fila de la tabla padre si hay filas en la tabla hija que la referencian (ej: no puedes eliminar un *Cliente* si tiene *Pedidos* asociados), a menos que se definan acciones específicas (como borrado en cascada, ¡peligroso!).
 - No se pueda cambiar el valor de la PK en la tabla padre si hay FKs apuntando a ella (a menos que se definan acciones específicas).
- **Ejemplo:**
 - Tabla *Clientes* (PK: *IdCliente*)
 - Tabla *Pedidos* (PK: *IdPedido*, FK: *IdCliente* que referencia a *Clientes(IdCliente)*).
 - El valor en la columna *IdCliente* de cada fila de *Pedidos* debe ser un *IdCliente* que exista en la tabla *Clientes*.

Parte 2: Modelando Relaciones y Asegurando la Calidad de los Datos

(2.1) Tipos de Relaciones entre Tablas

Las relaciones describen cuántas instancias de una entidad pueden estar asociadas con instancias de otra entidad. Hay tres tipos principales:

1. Relación Uno a Uno (1:1):

- **Descripción:** Cada fila en la Tabla A se relaciona con *ceros o una* fila en la Tabla B, y viceversa.
- **Ejemplo:** Un *Usuario* puede tener un *PerfilDetallado*. Un *PerfilDetallado* pertenece a un *Usuario*.
- **Implementación:** Generalmente, se coloca la PK de una tabla como FK en la otra. Para asegurar que sea 1:1 estricto, la columna FK en la segunda tabla también debería ser PK o tener una restricción **UNIQUE**. A menudo, si la relación es obligatoria, se considera simplemente añadir las columnas de B en la tabla A, a menos que haya muchas columnas opcionales o por razones de seguridad/rendimiento.

2. Relación Uno a Muchos (1:N):

- **Descripción:** Cada fila en la Tabla A (el lado "1") puede relacionarse con *ceros o muchas* filas en la Tabla B (el lado "N"), pero cada fila en la Tabla B se relaciona con *exactamente una* fila en la Tabla A. **Es el tipo de relación más común.**
- **Ejemplo:** Un *Cliente* (1) puede tener *muchos Pedidos* (N). Un *Pedido* (N) pertenece a un *Cliente* (1). Un *Autor* (1) puede escribir *muchos Libros* (N). Un *Libro* (N) es escrito por un *Autor* (asumiendo un solo autor por simplicidad aquí).
- **Implementación:** La **Clave Primaria (PK)** de la tabla del lado "1" (ej: *IdCliente*) se añade como **Clave Foránea (FK)** en la tabla del lado "N" (ej: *IdCliente* en la tabla *Pedidos*).

3. Relación Muchos a Muchos (N:N):

- **Descripción:** Cada fila en la Tabla A puede relacionarse con *ceros o muchas* filas en la Tabla B, y viceversa.

- *Ejemplo:* Un *Alumno* (N) puede cursar *muchas Materias* (N). Una *Materia* (N) puede ser cursada por *muchos Alumnos* (N). Un *Producto* (N) puede estar en *muchas Facturas* (N). Una *Factura* (N) puede contener *muchos Productos* (N).
- *Implementación:* **No se puede implementar directamente** con una sola FK. Se requiere una **tabla intermedia (o de unión, o asociativa)**.
 - Esta tabla de unión contendrá (como mínimo) **dos Claves Foráneas**: una que referencia a la PK de la Tabla A y otra que referencia a la PK de la Tabla B.
 - La **Clave Primaria** de la tabla de unión suele ser la **combinación de ambas FKs**.
 - La tabla de unión también puede contener atributos propios de la relación (ej: en una tabla *Inscripciones(IdAlumno, IdMateria)*, podría haber una columna *FechaInscripcion* o *NotaFinal*).
 - Ejemplo N:N Alumnos-Materias:
 - Tabla *Alumnos* (PK: *IdAlumno*)
 - Tabla *Materias* (PK: *IdMateria*)
 - Tabla *Cursadas* (PK: (*IdAlumno, IdMateria*), FK1: *IdAlumno* -> *Alumnos(IdAlumno)*, FK2: *IdMateria* -> *Materias(IdMateria)*, Otros Atributos: *Nota, Fecha*)

(2.2) Práctica de Modelado (Conceptual)

Antes de escribir SQL, es fundamental diseñar el modelo de datos. Usemos un ejemplo: **Una Biblioteca Simple**.

- **Entidades (Tablas):** *Libros, Autores, Socios, Prestamos*.
- **Atributos (Columnas) y Tipos:**
 - *Libros:* *IdLibro* (INT, PK), *Titulo* (VARCHAR), *ISBN* (VARCHAR, UNIQUE?), *IdAutor* (INT, FK a Autores), *AnioPublicacion* (INT).
 - *Autores:* *IdAutor* (INT, PK), *NombreAutor* (VARCHAR), *Nacionalidad* (VARCHAR).
 - *Socios:* *IdSocio* (INT, PK), *NombreSocio* (VARCHAR), *Direccion* (VARCHAR), *Telefono* (VARCHAR).
 - *Prestamos:* ¡Aquí está la relación N:N entre Libros y Socios! Un socio puede pedir muchos libros, un libro puede ser pedido por muchos socios (en distintos momentos). Necesitamos una tabla de unión.
 - *Prestamos:* *IdPrestamo* (INT, PK - artificial), *IdLibro* (INT, FK a Libros), *IdSocio* (INT, FK a Socios), *FechaPrestamo* (DATE), *FechaDevolucion* (DATE, permite NULL?).
- **Relaciones:**
 - Autores - Libros: 1:N (Un autor escribe muchos libros). FK *IdAutor* en *Libros*.
 - Socios - Prestamos: 1:N (Un socio realiza muchos préstamos). FK *IdSocio* en *Prestamos*.
 - Libros - Prestamos: 1:N (Un libro participa en muchos préstamos). FK *IdLibro* en *Prestamos*.
- **Diagrama (DER simplificado):** Se dibujaría en la pizarra mostrando las tablas como cajas y las relaciones como líneas conectando PKs y FKs, indicando la cardinalidad (1, N).

(2.3) Normalización: Buscando la Calidad del Diseño

Una vez que tenemos un modelo inicial, aplicamos la **normalización** para mejorar su calidad.

- *¿Por qué Normalizar?*
 - **Minimizar la Redundancia:** Evitar almacenar la misma información varias veces innecesariamente.
 - **Mejorar la Integridad:** Asegurar que los datos sean consistentes y válidos.

- **Evitar Anomalías:** Prevenir problemas al insertar, actualizar o borrar datos.
- **Formas Normales (Concepto y Propósito):** Son un conjunto de reglas que nos guían para estructurar las tablas. Nos enfocaremos en las tres primeras:
 1. **Primera Forma Normal (1FN):**
 - *Regla:* Cada celda de la tabla debe contener un **valor único y atómico** (indivisible). No debe haber **grupos repetitivos** dentro de una fila. Toda tabla debe tener una **Clave Primaria**.
 - *Ejemplo de Violación:* Una columna *Telefonos* que almacena "11-222-333, 11-444-555". Se viola la atomicidad.
 - *Solución:* Crear una tabla separada *TelefonosSocio(IdSocio, Telefono)* con una relación 1:N.
 - *Ejemplo de Violación:* Columnas *Hijo1_Nombre*, *Hijo1_FechaNac*, *Hijo2_Nombre*, *Hijo2_FechaNac*. Es un grupo repetitivo.
 - *Solución:* Crear una tabla separada *HijosSocio(IdHijo, IdSocio, Nombre, FechaNac)*.
 - *En resumen:* Tablas bien definidas con PK y sin valores "múltiples" en una celda. Es el punto de partida básico.
 2. **Segunda Forma Normal (2FN):**
 - *Regla:* La tabla debe estar en 1FN **y** todos los atributos que **no** forman parte de la Clave Primaria deben depender funcionalmente de la **totalidad** de la Clave Primaria.
 - *Relevancia:* Principalmente cuando la PK es **compuesta** (formada por más de una columna).
 - *Ejemplo de Violación:* Tabla *DetallePedido(IdPedido, IdProducto, DescripcionProducto, Cantidad, PrecioUnitario)*. Si la PK es (*IdPedido, IdProducto*), entonces *Cantidad* y *PrecioUnitario* dependen de *ambas* partes de la PK. Pero *DescripcionProducto* depende *solo* de *IdProducto*. Esto es una **dependencia parcial**.
 - *Problema:* La descripción del producto se repetiría en cada línea de pedido para ese producto (redundancia) y si cambia la descripción, habría que actualizarla en muchos lugares (anomalía de actualización).
 - *Solución:* Dividir la tabla. Mantener *DetallePedido(IdPedido, IdProducto, Cantidad, PrecioUnitario)* y crear una tabla *Productos(IdProducto, DescripcionProducto)*.
 3. **Tercera Forma Normal (3FN):**
 - *Regla:* La tabla debe estar en 2FN **y** no debe existir ninguna **dependencia transitiva** entre atributos no-clave. Es decir, un atributo no-clave no debe depender funcionalmente de otro atributo no-clave.
 - *Ejemplo de Violación:* Tabla *Empleados(IdEmpleado, Nombre, IdDepartamento, NombreDepartamento, UbicacionDepartamento)*. Si la PK es *IdEmpleado*, y *NombreDepartamento* y *UbicacionDepartamento* dependen de *IdDepartamento* (que no es clave), entonces hay una dependencia transitiva (*IdEmpleado* -> *IdDepartamento* -> *NombreDepartamento*).
 - *Problema:* El nombre y ubicación del departamento se repetirían para cada empleado de ese departamento (redundancia), y si cambia el nombre del departamento, hay que actualizarlo en muchos registros (anomalía).
 - *Solución:* Dividir la tabla. Mantener *Empleados(IdEmpleado, Nombre, IdDepartamento)* y crear una tabla *Departamentos(IdDepartamento, NombreDepartamento, UbicacionDepartamento)*.
- **Objetivo Práctico:** Para la mayoría de las aplicaciones, alcanzar la 3FN proporciona un diseño robusto y eficiente. La normalización implica a menudo crear más tablas, pero esto generalmente es bueno para

la integridad, aunque puede requerir más uniones (*JOINS*) al consultar (lo veremos después).

(2.4) Herramientas de SQL Server: SSMS y Azure Data Studio

Para interactuar con nuestra base de datos SQL Server, necesitamos una herramienta cliente. Las más comunes de Microsoft son:

- **SQL Server Management Studio (SSMS):** La herramienta clásica, muy completa, solo para Windows.
- **Azure Data Studio:** Más moderna, ligera, multiplataforma (Windows, macOS, Linux), con mejor soporte para otras bases de datos (además de SQL Server) y extensiones. **Recomendada si tienes opción.**
- **Conexión:** Al abrir cualquiera de ellas, necesitas conectarte a tu instancia de SQL Server. Para la versión Express instalada localmente, generalmente usas:
 - *Server name:* `localhost\SQLEXPRESS` o `(localdb)\MSSQLLocalDB` (depende de cómo se instaló) o simplemente `.` (punto).
 - *Authentication:* `Windows Authentication` (usa tu usuario de Windows).
- **Interfaz (similar en ambas):**
 - **Explorador de Objetos (Object Explorer / Servers):** Panel lateral (usualmente izquierda) que muestra una vista de árbol de los servidores conectados, sus bases de datos, y dentro de cada base de datos: Tablas, Vistas, Procedimientos Almacenados, etc. Puedes expandir las tablas para ver Columnas, Claves, Índices.
 - **Editor de Consultas (Query Editor):** El área principal donde escribes tus comandos SQL. Puedes tener varias pestañas abiertas. Presionas un botón (Ejecutar/Execute o F5) para enviar el SQL al servidor.
 - **Resultados (Results):** Panel inferior donde se muestra la salida de los comandos *SELECT* en forma de tabla.
 - **Mensajes (Messages):** Otro panel inferior que muestra información sobre la ejecución de los comandos (éxito, errores, número de filas afectadas).

Parte 3: Definiendo la Estructura - SQL DDL (Data Definition Language)

Ahora que entendemos los conceptos, vamos a usar SQL para crear la estructura de nuestra base de datos. El subconjunto de SQL para esto se llama **DDL**.

(3.1) Introducción a SQL

- **SQL (Structured Query Language):** Es el lenguaje estándar para gestionar y consultar bases de datos relacionales.
- **Declarativo:** Le dices a la base de datos *qué* resultado quieres, no los pasos exactos de cómo conseguirlo (el DBMS optimiza eso).
- **Subconjuntos:**
 - *DDL:* Definir datos (estructuras). `CREATE`, `ALTER`, `DROP`.
 - *DML:* Manipular datos (contenido). `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
 - *DCL:* Controlar acceso (`GRANT`, `REVOKE`).
 - *TCL:* Controlar transacciones (`COMMIT`, `ROLLBACK`).

(3.2) Comandos DDL Esenciales

- `CREATE DATABASE NombreDB;`
 - ¿Qué hace? Crea una nueva base de datos vacía en el servidor SQL Server al que estás conectado.
 - Ejemplo: `CREATE DATABASE BibliotecaDB;`

- **USE NombreDB;**
 - ¿Qué hace? Cambia el "contexto" de tu sesión actual a la base de datos especificada. ¡Debes ejecutar esto antes de intentar crear tablas dentro de esa base de datos!
 - Ejemplo: **USE BibliotecaDB;**
- **CREATE TABLE NombreTabla (...);**
 - ¿Qué hace? Crea una nueva tabla dentro de la base de datos activa.
 - Sintaxis Detallada:

```
CREATE TABLE NombreTabla (
    NombreColumna1 TipoDato [CONSTRAINT NombreC1 ConstraintColumna1
...],
    NombreColumna2 TipoDato [CONSTRAINT NombreC2 ConstraintColumna2
...],
    -- ... más columnas ...
    [CONSTRAINT NombreCT1 ConstraintTabla1 ...],
    [CONSTRAINT NombreCT2 ConstraintTabla2 ...]
    -- ... más constraints de tabla ...
);
```

- Ejemplo (Tabla Autores):

```
CREATE TABLE Autores (
    IdAutor INT CONSTRAINT PK_Autores PRIMARY KEY IDENTITY(1,1), -- PK
    artificial, autoincremental
    NombreAutor NVARCHAR(100) NOT NULL,
    Nacionalidad VARCHAR(50) NULL -- Permite NULLs
);
```

- **IDENTITY(1,1):** Especifica que *IdAutor* es autoincremental, empezando en 1 y aumentando de 1 en 1. SQL Server gestiona el valor automáticamente al insertar.
- **Constraints (Restricciones): Garantizando la Integridad**
 - ¿Por qué son vitales? Delegan la responsabilidad de mantener la integridad de los datos al DBMS, en lugar de depender de que la aplicación siempre envíe datos correctos. Son reglas definidas a nivel de base de datos.
 - **Tipos y Sintaxis:**
 - **PRIMARY KEY:** Define la clave primaria.
 - Nivel Columna (simple): **IdAutor INT PRIMARY KEY IDENTITY(1,1)**
 - Nivel Tabla (simple o compuesta, nombrada): **CONSTRAINT PK_Autores PRIMARY KEY (IdAutor)** o **CONSTRAINT PK_Inscripcion PRIMARY KEY (IdAlumno, IdMateria)**
 - **FOREIGN KEY:** Define una clave foránea. **Se recomienda definirla a nivel de tabla y nombrarla.**
 - Nivel Tabla: **CONSTRAINT FK_Libros_Autores FOREIGN KEY (IdAutor) REFERENCES Autores(IdAutor)**
 - **NOT NULL:** La columna no puede quedar vacía.
 - Nivel Columna: **NombreAutor NVARCHAR(100) NOT NULL**

- **UNIQUE**: El valor debe ser único en la columna (o combinación de columnas). Permite *un solo* valor NULL (a diferencia de PK).
 - Nivel Columna: `Email VARCHAR(255) UNIQUE`
 - Nivel Tabla: `CONSTRAINT UQ_Socios_Email UNIQUE (Email)`
- **CHECK**: Valida una condición lógica.
 - Nivel Columna: `Precio DECIMAL(10, 2) CHECK (Precio >= 0)`
 - Nivel Tabla: `CONSTRAINT CK_FechasPrestamo CHECK (FechaDevolucion IS NULL OR FechaDevolucion >= FechaPrestamo)`
- **DEFAULT**: Asigna un valor por defecto si no se especifica al insertar.
 - Nivel Columna: `FechaAlta DATE DEFAULT GETDATE()` (`GETDATE()` es una función de SQL Server que devuelve la fecha/hora actual).
- *Ejemplo (Tabla Libros con FK a Autores):*

```
CREATE TABLE Libros (
  IdLibro INT CONSTRAINT PK_Libros PRIMARY KEY IDENTITY(1,1),
  Titulo NVARCHAR(200) NOT NULL,
  ISBN VARCHAR(20) CONSTRAINT UQ_Libros_ISBN UNIQUE, -- ISBN debe ser
  único
  AnioPublicacion INT NULL,
  IdAutor INT NOT NULL, -- Columna para la FK (debe ser NOT NULL si
  un libro siempre tiene autor)

  CONSTRAINT FK_Libros_Autores -- Nombre de la FK
  FOREIGN KEY (IdAutor) -- Columna(s) en esta tabla
  REFERENCES Autores(IdAutor) -- Tabla y Columna(s) referenciadas
  -- ON DELETE NO ACTION -- Acción por defecto si se intenta
  borrar un autor con libros
  -- ON UPDATE NO ACTION -- Acción por defecto si se intenta
  cambiar IdAutor en Autores
);
```

- **ALTER TABLE NombreTabla ... ;**
 - ¿Qué hace? Modifica la estructura de una tabla existente. Útil, pero es mejor diseñar bien desde el principio.
 - Operaciones Comunes:
 - `ADD NombreColumna TipoDato [Constraint];` (Añadir columna)
 - `DROP COLUMN NombreColumna;` (Quitar columna)
 - `ALTER COLUMN NombreColumna NuevoTipoDato [NULL|NOT NULL];` (Cambiar tipo/nulabilidad)
 - `ADD CONSTRAINT ...;` (Añadir restricción)
 - `DROP CONSTRAINT NombreConstraint;` (Quitar restricción)
 - *Ejemplo:* `ALTER TABLE Libros ADD CantidadPaginas INT NULL;`
- **DROP TABLE NombreTabla;**
 - ¿Qué hace? **Elimina permanentemente** la tabla y todos los datos que contiene. No hay "papelera de reciclaje". ¡Usar con extrema precaución!
- **DROP DATABASE NombreBaseDatos;**

- ¿Qué hace? **Elimina permanentemente** toda la base de datos, incluyendo todas sus tablas, datos y otros objetos. ¡Aún más peligroso!

(3.3) Práctica DDL

El objetivo es traducir el modelo de datos diseñado (ej: Biblioteca) a un script SQL (`.sql`) que contenga las sentencias `CREATE TABLE` para todas las tablas, incluyendo la definición de columnas con tipos de datos apropiados y las restricciones `PRIMARY KEY`, `FOREIGN KEY` y `NOT NULL` (y opcionalmente `UNIQUE` o `CHECK` donde aplique). Luego, ejecutar este script en SSMS/Azure Data Studio dentro de la base de datos creada (`USE BibliotecaDB`; al principio).

Parte 4: Trabajando con los Datos - SQL DML (Data Manipulation Language)

Una vez creada la estructura con DDL, usamos DML para interactuar con los datos *dentro* de las tablas.

(4.1) Insertando Datos (`INSERT INTO`)

- ¿Qué hace? Añade nuevas filas a una tabla.
- *Sintaxis Recomendada (listando columnas):*

```
INSERT INTO NombreTabla (Columna1, Columna2, Columna3)
VALUES (Valor1, Valor2, Valor3);
```

- Los valores deben coincidir en orden y tipo con las columnas listadas.
- Las columnas omitidas tomarán su valor `DEFAULT` (si existe) o `NULL` (si la columna lo permite). Si una columna es `NOT NULL` y no tiene `DEFAULT`, ¡debes incluirla!
- *Ejemplo (insertando un autor):*

```
INSERT INTO Autores (NombreAutor, Nacionalidad)
VALUES (N'Gabriel García Márquez', 'Colombiano'); -- N' para strings Unicode (NVARCHAR)
```

- *Insertando Múltiples Filas:*

```
INSERT INTO Socios (NombreSocio, Direccion, Telefono) VALUES
('Ana Lopez', 'Calle Falsa 123', '11-1111-1111'),
('Juan Perez', 'Av. Siempre Viva 742', '11-2222-2222'),
('Maria Elena', NULL, '11-3333-3333'); -- Dirección es NULL
```

(4.2) Consultando Datos (`SELECT`) - La Joya de SQL

- ¿Qué hace? Recupera datos de una o más tablas. Es el comando más versátil y utilizado.
- **Sintaxis Básica:** `SELECT lista_de_columnas FROM nombre_tabla;`
 - `SELECT NombreAutor, Nacionalidad FROM Autores;` (Selecciona columnas específicas).
 - `SELECT * FROM Libros;` (Selecciona **todas** las columnas. Útil para explorar, pero **evitar en aplicaciones** porque es ineficiente y puede romperse si la tabla cambia).

- **Filtrando Filas (WHERE):** Permite seleccionar solo las filas que cumplen una condición.
 - *Sintaxis:* `SELECT ... FROM ... WHERE condicion_logica;`
 - *Condiciones Comunes:*
 - Comparación: `WHERE AnioPublicacion > 1990, WHERE Nacionalidad = 'Argentino', WHERE Precio != 0`
 - Lógicas: `WHERE Nacionalidad = 'Argentino' AND AnioPublicacion > 2000, WHERE IdAutor = 1 OR IdAutor = 5`
 - Rango: `WHERE Precio BETWEEN 1500 AND 3000` (incluye límites)
 - Lista: `WHERE IdAutor IN (1, 3, 7)` (es igual a `IdAutor=1 OR IdAutor=3 OR IdAutor=7`)
 - Nulos: `WHERE FechaDevolucion IS NULL, WHERE Telefono IS NOT NULL`
 - Patrones de Texto (LIKE):
 - %: Comodín para cero o más caracteres. `WHERE Titulo LIKE 'E1%'` (empieza con "E"). `WHERE Titulo LIKE '%Guerra%'` (contiene "Guerra"). `WHERE Titulo LIKE '%s'` (termina con "s").
 - _: Comodín para un solo carácter. `WHERE CodigoPostal LIKE 'B171_AA'`
 - *Ejemplo:* `SELECT Titulo, AnioPublicacion FROM Libros WHERE IdAutor = 1 ORDER BY AnioPublicacion DESC;`
- **Ordenando Resultados (ORDER BY):** Controla el orden en que se muestran las filas.
 - *Sintaxis:* `SELECT ... FROM ... [WHERE ...] ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC] ...;`
 - *ASC:* Ascendente (A-Z, 0-9). Es el valor por defecto si no se especifica.
 - *DESC:* Descendente (Z-A, 9-0).
 - Se puede ordenar por múltiples columnas.
 - *Ejemplo:* `SELECT NombreSocio, Direccion FROM Socios ORDER BY NombreSocio ASC;`
- **Uniendo Tablas (INNER JOIN):** Combina filas de dos (o más) tablas relacionadas.
 - ¿Por qué? Para obtener información que está distribuida en varias tablas (gracias a la normalización). Ej: obtener el título del libro y el nombre de su autor.
 - *INNER JOIN:* Devuelve **solo** las filas para las cuales la condición de unión (generalmente PK = FK) se cumple en **ambas** tablas. Si un libro tiene un *IdAutor* que no existe en la tabla *Autores* (no debería pasar si la FK está bien definida), ese libro no aparecerá en el resultado del *INNER JOIN*.
 - *Sintaxis:*

```
SELECT
    l.Titulo, -- Usar alias.columna
    a.NombreAutor
FROM
    Libros l -- Alias 'l' para Libros
INNER JOIN
    Autores a ON l.IdAutor = a.IdAutor -- Condición de unión PK = FK
WHERE
    a.Nacionalidad = 'Colombiano'
ORDER BY
    l.Titulo;
```

- **Alias de Tabla (l, a):** Son cruciales para acortar la sintaxis y, sobre todo, para desambiguar si ambas tablas tuvieran una columna con el mismo nombre (ej: *Id*). Se definen después del

nombre de la tabla en el *FROM* o *JOIN*.

(4.3) Modificando Datos (**UPDATE**)

- ¿Qué hace? Cambia los valores en filas existentes que cumplen una condición.
- *Sintaxis*:

```
UPDATE NombreTabla
SET Columna1 = NuevoValor1,
    Columna2 = NuevoValor2
    -- ... más columnas ...
WHERE Condicion; -- ¡¡¡LA CLÁUSULA WHERE ES FUNDAMENTAL!!!
```

- **¡PELIGRO!** Si olvidas el *WHERE*, la cláusula *SET* se aplicará a **TODAS LAS FILAS** de la tabla. Siempre prueba tu condición *WHERE* con un *SELECT* primero para asegurarte de que selecciona exactamente las filas que quieres modificar.
- *Ejemplo*: `UPDATE Libros SET AnioPublicacion = 1968 WHERE IdLibro = 5;`

(4.4) Eliminando Datos (**DELETE**)

- ¿Qué hace? Borra filas completas de una tabla que cumplen una condición.
- *Sintaxis*:

```
DELETE FROM NombreTabla
WHERE Condicion; -- ¡¡¡LA CLÁUSULA WHERE ES FUNDAMENTAL!!!
```

- **¡PELIGRO!** Si olvidas el *WHERE*, ¡borrarás **TODAS LAS FILAS** de la tabla! Es aún más peligroso que *UPDATE*. Siempre prueba tu *WHERE* con un *SELECT ** primero. Además, las restricciones de Clave Foránea pueden impedir el borrado si otras tablas dependen de las filas que intentas eliminar.
- *Ejemplo*: `DELETE FROM Socios WHERE IdSocio = 10;` (Solo si no tiene préstamos asociados, por ejemplo).

Conclusión

Comprender el modelo relacional y saber utilizar SQL para definir estructuras (DDL) y manipular datos (DML) es una habilidad esencial para cualquier programador. Un buen diseño de base de datos (normalizado) y el uso correcto y seguro de SQL son la base para construir aplicaciones robustas y eficientes que manejen información de manera confiable. La práctica constante escribiendo y ejecutando consultas es la mejor forma de dominar este lenguaje.

Síntesis de una oración: Este material cubre los fundamentos del modelo relacional de bases de datos, el diseño mediante normalización y relaciones (PK/FK), y el uso del lenguaje SQL (DDL para estructuras y DML para datos) con SQL Server y herramientas como SSMS/Azure Data Studio.

Síntesis de 5 puntos importantes:

1. El **Modelo Relacional** organiza datos en **Tablas** con **Columnas** (tipos de datos) y **Filas**, usando **Claves Primarias (PK)** para identificación única y **Claves Foráneas (FK)** para establecer relaciones (1:1, 1:N, N:N con tabla de unión) y mantener integridad referencial.
2. La **Normalización** (1FN, 2FN, 3FN) es un proceso de diseño para minimizar redundancia y evitar anomalías de datos.
3. **SQL** es el lenguaje estándar; **DDL** (*CREATE*, *ALTER*, *DROP*) define la estructura y **Constraints** (PK, FK, NOT NULL, UNIQUE, CHECK) imponen reglas de integridad.
4. **DML** manipula datos: *INSERT* añade filas, *UPDATE* modifica filas existentes (¡cuidado con *WHERE*!), *DELETE* elimina filas (¡cuidado con *WHERE*!).
5. *SELECT* es el comando DML clave para consultar datos, permitiendo filtrar (*WHERE*), ordenar (*ORDER BY*) y combinar información de múltiples tablas (*INNER JOIN*).

49583